

## LECTURE 12

# The Entity-Relationship Model

Entities, relationships, and attributes — before a single table is created

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

## What This Lecture Covers

- 1 Entity types and entity occurrences
- 2 Relationship types and relationship occurrences
- 3 Five attribute categories: simple, composite, single-valued, multi-valued, derived
- 4 Strong entity types vs. weak entity types
- 5 Attributes that belong to a relationship itself
- 6 Structural constraints, then a 9-step worked ER walkthrough: university enrollment, entity by entity

## Entities, Relationships, Attributes

- The ER model represents an organization's data as entities (things worth tracking), relationships (meaningful associations), and attributes (properties of either).
- Introduced by Peter Chen in 1976 to give conceptual modeling a precise, drawable notation both business stakeholders and designers could read.

### Branch

branchNo

street

city

postcode

*An entity type — the basic building block*

## Entity Types and Occurrences

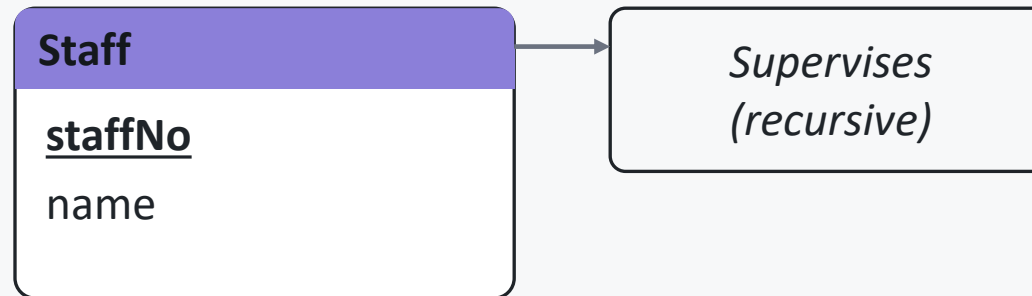
- An entity type is a group of objects with the same properties, significant enough to track independently — Staff, Branch, and PropertyForRent are all entity types.
- An entity occurrence (or entity instance) is one uniquely identifiable member of an entity type — the specific staff member SL21, John White, is one occurrence of the Staff entity type.

### NOTE

Entity type vs. relation — same box, different lifecycle. The entity type Branch is a conceptual, technology-free idea; the Branch relation (Lecture 11) is the logical mapping of that idea, complete with a chosen primary key and eventual data types.

## Binary, Recursive, and Ternary Relationships

- A relationship type is a meaningful association among entity types. A relationship occurrence is one specific, uniquely identifiable association — “SL21 works at B005” is one occurrence of Has.
- Most relationships in this course are binary — they connect exactly two entity types, like Has connecting Branch and Staff.
- A relationship connecting an entity type to itself is recursive (unary); one connecting three entity types at once is ternary.



*“A Staff member Supervises another Staff member” — both participating occurrences come from the same entity type.*

## Every Attribute Category, In One Entity

### PrivateOwner

#### ownerNo

name

address

telNo {}

*numberOfProperties*

- Simple (atomic): ownerNo — cannot be meaningfully subdivided.
- Composite: name, address — divisible into meaningful sub-parts.
- Single-valued: ownerNo — exactly one value per occurrence.
- Multi-valued (telNo {}): can hold more than one value per occurrence.
- Derived (numberOfProperties): computed from other data, never stored — storing it would create exactly the redundancy Lecture 11 warned about.

## Attribute Categories at a Glance

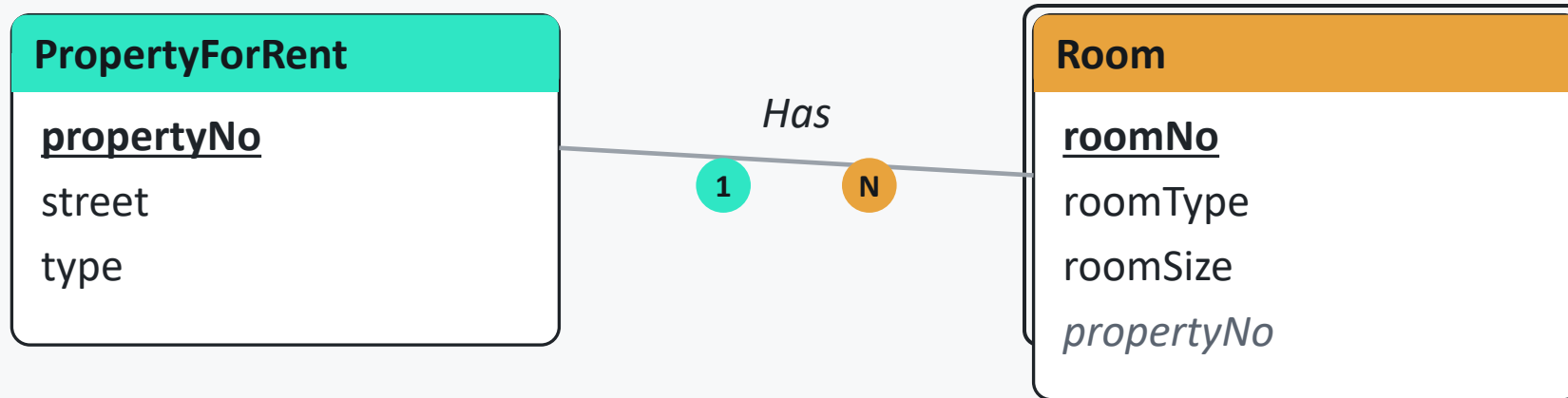
| Axis         | Category         | Meaning                                               |
|--------------|------------------|-------------------------------------------------------|
| Divisibility | Simple (atomic)  | Cannot be meaningfully subdivided further             |
| Divisibility | Composite        | Can be divided into meaningful sub-parts              |
| Value count  | Single-valued    | Holds exactly one value per occurrence                |
| Value count  | Multi-valued { } | Can hold more than one value per occurrence           |
| Storage      | Derived          | Computed from other attributes, never stored directly |

**WARNING**

A multi-valued attribute cannot be stored as one column. Mapping telNo { } to the relational model does not produce one column, but a whole new relation, OwnerTelNo(ownerNo, telNo), with one row per phone number.

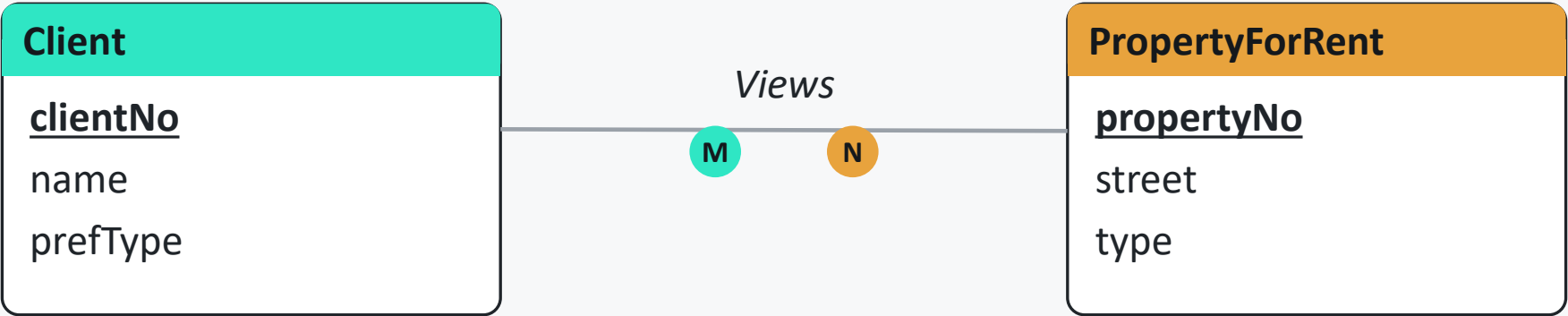
## Weak Entity Types

- A strong entity type has independent existence and its own key uniquely identifies every occurrence — Branch, Staff, PropertyForRent, PrivateOwner.
- A weak entity type depends on an owner entity type, and its own key is only a partial key — unique within one owner occurrence, not across the whole entity type.



*Room is a weak entity — double border, existence depends on PropertyForRent. True key: (propertyNo, roomNo).*

# Attributes That Belong to the Pairing



## Views carries its own attributes

| Attribute of Views | Meaning                                                     |
|--------------------|-------------------------------------------------------------|
| viewDate           | The date this specific client viewed this specific property |
| comment            | Feedback left after that specific viewing                   |

Neither attribute belongs to Client alone or PropertyForRent alone — each only makes sense attached to one pairing. This is why an M:N relationship with attributes maps to its own relation: Viewing(clientNo, propertyNo, viewDate, comment).

## Cardinality (max) + Participation (min)

- Drawing M and N next to a relationship diamond communicates how many occurrences can pair up — but says nothing about whether pairing up is required.
- A complete description needs both pieces, together called structural constraints, usually written as a (min, max) pair for each participating entity type.

### **max** Cardinality

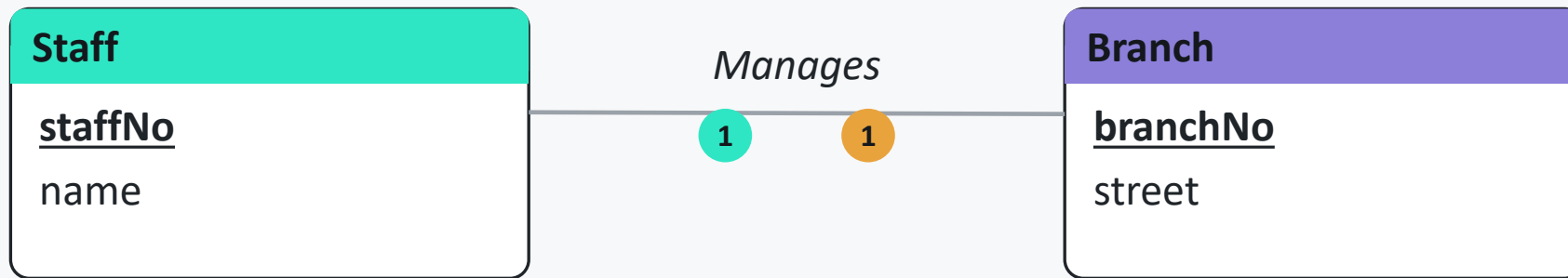
The maximum number of relationship occurrences an entity occurrence can participate in.

### **min** Participation

The minimum number of relationship occurrences an entity occurrence must participate in (0 or 1, almost always).

## One-to-One (1:1)

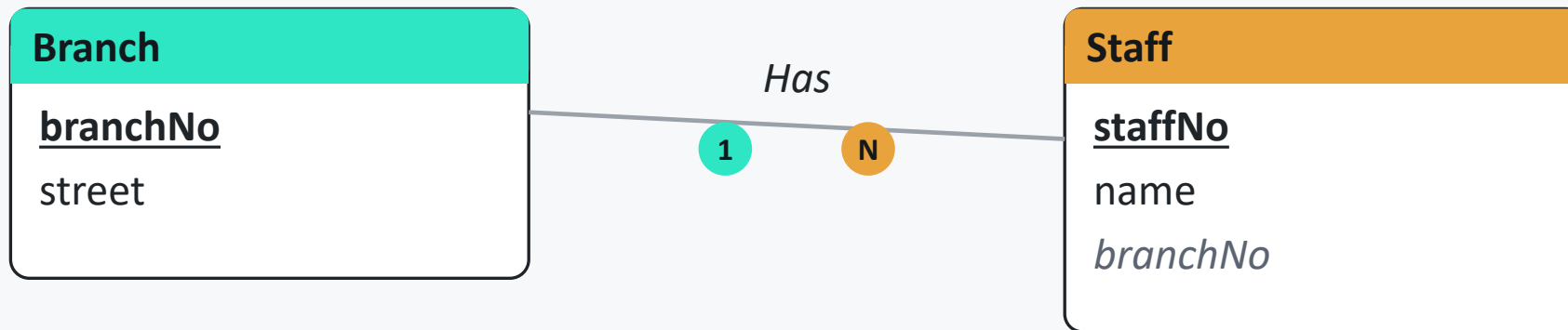
Every occurrence on each side is associated with at most one occurrence on the other side.



*A Staff member manages at most one Branch, and a Branch is managed by at most one Staff member.*

## One-to-Many (1:N)

One occurrence on one side can be associated with many on the other, but each of those many relates back to only one.



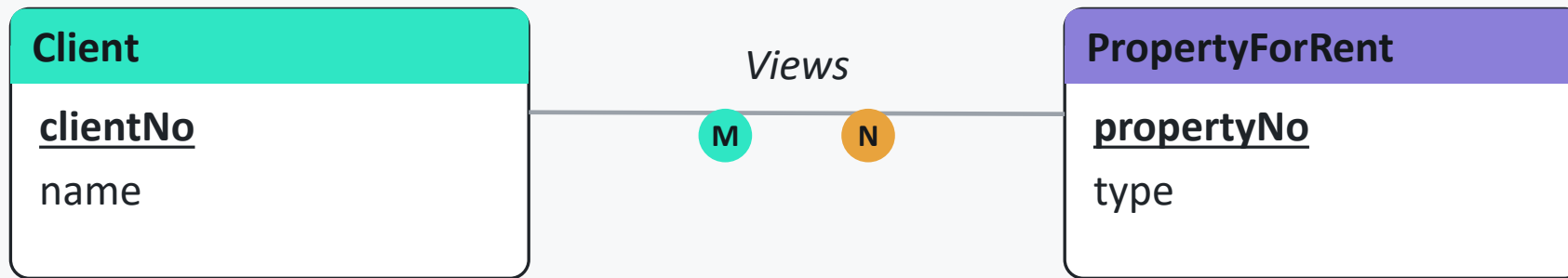
*One Branch employs many Staff, but each Staff member works at exactly one Branch.*

**TIP**

Look across: the N next to Staff means one Branch connects to many Staff.

## Many-to-Many (M:N)

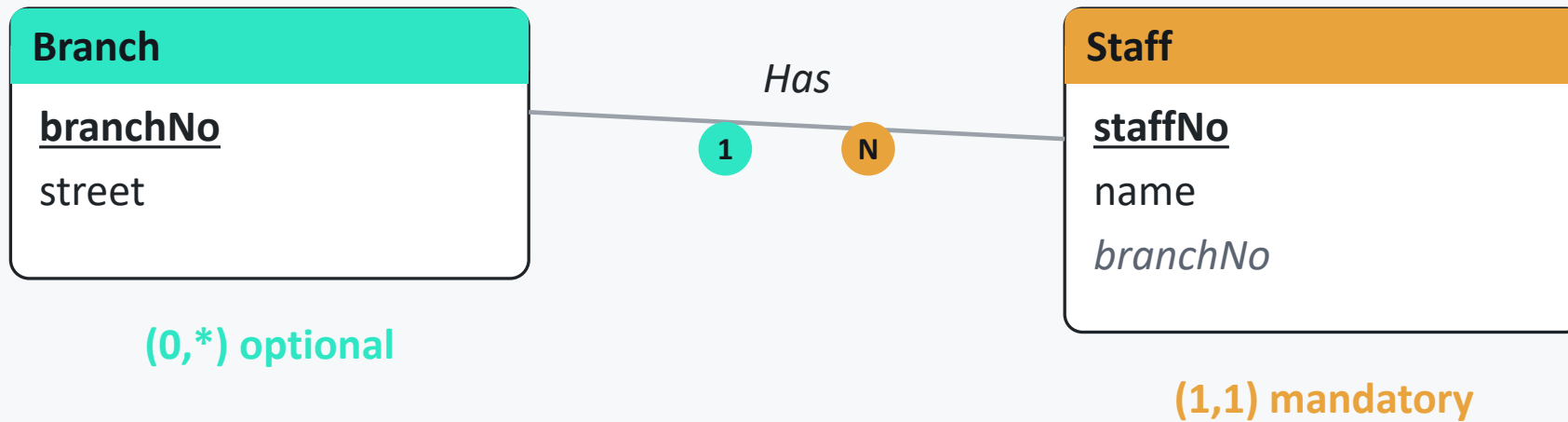
Occurrences on either side can be associated with many occurrences on the other.



*Many Clients can view many PropertyForRent listings, and each property can be viewed by many different clients.*

## Mandatory vs. Optional Participation

- Mandatory (total): every occurrence of the entity type must appear in at least one relationship occurrence; min = 1. Drawn with a double line.
- Optional (partial): an occurrence is allowed to appear in zero relationship occurrences; min = 0. Drawn with a single line.



*Staff.branchNo is never NULL — every staff member must work at exactly one branch. A newly opened Branch may legitimately have zero staff assigned yet.*

## Cardinality and Participation Are Independent

### WARNING

It is tempting to read “1:N” and assume the “1” side is automatically mandatory. It is not. Cardinality answers “how many can there be?” (the 1, N and M badges). Participation answers “must there be at least one?” (mandatory or optional). In Has, Staff is mandatory (1,1) because every staff member needs a branch, while Branch is optional (0,N) because a new branch can have no staff yet.

A relationship needs both answered, for each side, to be fully specified — that's exactly why they're both listed together as one (min, max) pair.

## Start From Plain-English Requirements

- Scenario: a university enrollment system. (R1) A student belongs to exactly one department. (R2) A department offers many courses, and each course is offered by exactly one department. (R3) Students enroll in courses, and each enrollment records a grade.
- (R4) A course is split into sections numbered 1, 2, 3 within each course, and section numbers restart in every course. (R5) A section cannot exist without its course.

### NOTE

The design never starts from tables. Every noun and verb in these five requirements is a candidate for an entity, an attribute or a relationship; the next steps decide which.

# Step 2: Sort Each Noun

| Noun       | Verdict                | Reason                                                                     |
|------------|------------------------|----------------------------------------------------------------------------|
| Student    | Entity                 | Own attributes (studentId, name, email); many occurrences; exists alone.   |
| Department | Entity                 | Own attributes (deptCode, deptName); shared by many students.              |
| Course     | Entity                 | Own attributes (courseCode, title, creditHours); exists before enrollment. |
| Section    | Weak entity            | Meaningful only inside one course; sectionNo repeats across courses.       |
| Email      | Attribute              | One value per student; describes only that student.                        |
| Grade      | Relationship attribute | Belongs to a (student, course) pairing, not to either alone.               |
| Term       | Attribute of Section   | Describes the section, not a standalone thing.                             |

**WARNING**

Do not promote every noun to an entity. A noun that only describes one thing (email, term) is an attribute; a noun that depends on a pairing of two things (grade) is a relationship attribute.

## Why Department Is an Entity, Not an Attribute

- Test 1: does it have attributes of its own? Yes: deptCode, deptName, building. Storing the department as a plain attribute of Student would copy its name into every student row.
- Test 2: does it exist without the other entity? Yes: a department exists before any student joins it, so it needs its own occurrences and its own key.

### RULE OF THUMB

If a noun has several attributes of its own and is shared by many occurrences of another entity, make it an entity. A single fact about one thing is an attribute.

## Why Grade Belongs on the Relationship

- Try Student: a student takes many courses, so one student row would need many grades. That fails.
- Try Course: a course has many students, each with a different grade. That fails too. A grade belongs to one specific pairing: this student in this course.

### RULE OF THUMB

The same reasoning gives viewDate and comment on Views. A fact that needs both participants named becomes a relationship attribute, and the M:N relationship maps to its own relation: Enrollment(studentId, courseCode, grade).

# Step 5: Turn Verbs Into Relationships

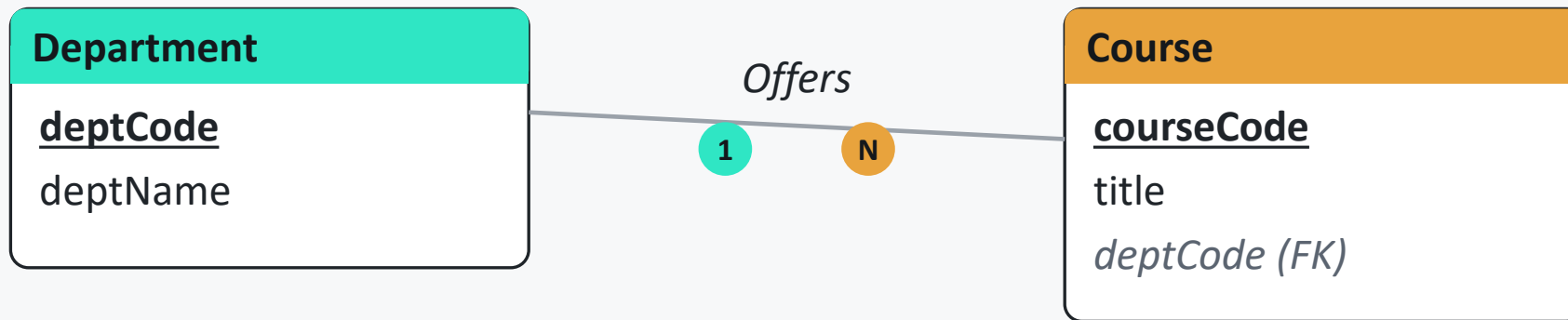
| Verb         | Participants           | Cardinality                                 |
|--------------|------------------------|---------------------------------------------|
| belongs to   | Student and Department | 1:N (each student has one department)       |
| offers       | Department and Course  | 1:N (each course has one department)        |
| enrolls in   | Student and Course     | M:N, with grade as a relationship attribute |
| has sections | Course and Section     | 1:N, identifying (Section is weak)          |

**TIP**

Each verb that links two entity types becomes a relationship. The cardinality comes from asking two questions in both directions, shown on the next slides.

## Step 6: Ask Two Questions About Offers

Question A: for one course, how many departments offer it? Exactly one. Question B: for one department, how many courses does it offer? Many. So Offers is 1:N, with Department on the one side and Course on the many side.



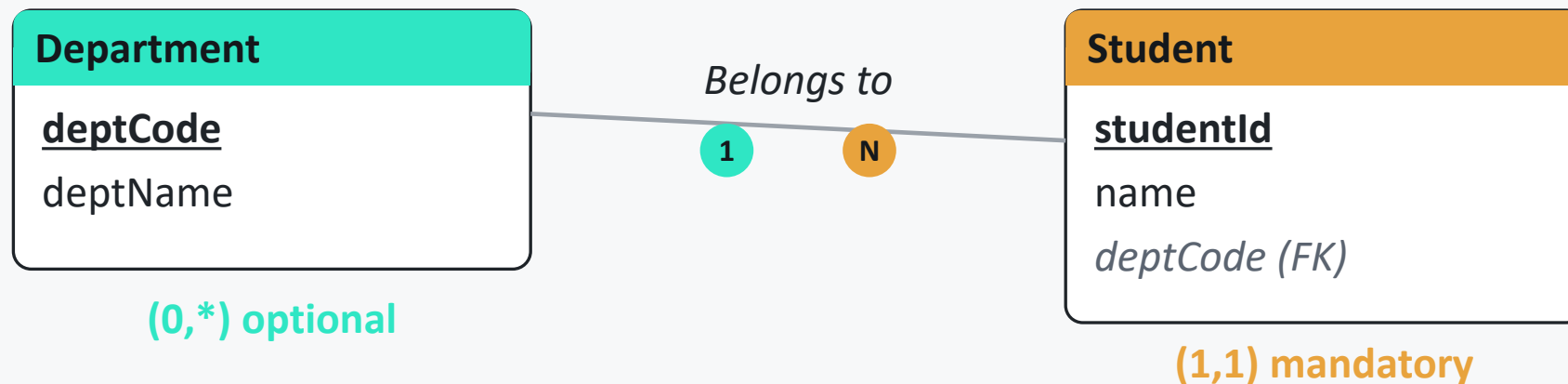
*One Department offers many Courses, but each Course is offered by exactly one Department.*

**TIP**

Read the badges in plain English first: 1 next to Department, N next to Course.

## Step 7: Who Must Take Part?

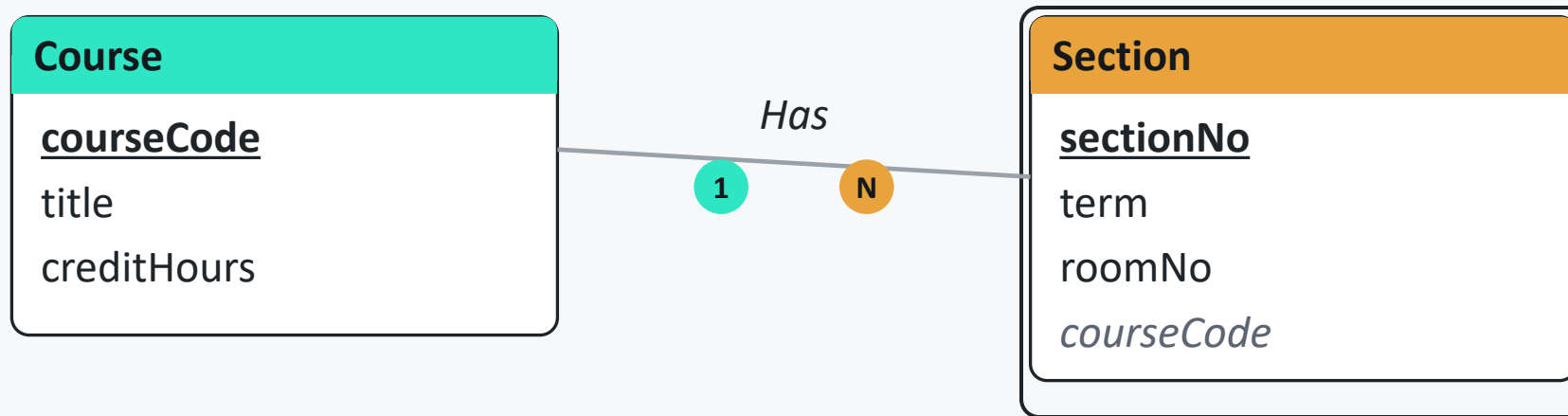
- Mandatory: every student must belong to exactly one department, so Student's participation in Belongs to is (1,1).
- Optional: a new department may have no students yet, so Department's participation is (0,\*).



*Student.deptCode is never NULL: every student must belong to exactly one department. A department with no students is still a valid fact, so Department stays optional.*

## Step 8: Section Is a Weak Entity

- Section depends on Course: section 1 exists in every course, so sectionNo alone cannot identify a section.
- Its true key is (courseCode, sectionNo): courseCode is the owner's key, and sectionNo is only the partial key.



*Section is a weak entity: double border, and its existence depends on Course. True key: (courseCode, sectionNo).*

## Why Section Cannot Be a Strong Entity

- Existence test: can a section exist without its course? No. Section 2 of CS101 means nothing once CS101 is removed, so Section fails the test a strong entity must pass.
- Uniqueness test: is sectionNo unique on its own? No. Sections 1 and 2 exist in every course. Only the owner's key plus the partial key identifies one section.

### CONTRAST

Course is strong: it exists in the catalogue before any section is scheduled, and courseCode alone identifies it.

## KEY TAKEAWAYS

# The Entity-Relationship Model

- 1 The ER model represents data as entity types (things), relationship types (associations), and attributes (properties).
- 2 Attributes divide along two axes plus one extra: simple vs. composite, single-valued vs. multi-valued, and derived (computed, never stored).
- 3 A strong entity type has independent existence and a key that identifies it alone; a weak entity type depends on an owner entity and has only a partial key.
- 4 A relationship type can carry its own attributes when a fact belongs to the pairing, not to either participant — mapping it to its own relation.
- 5 Structural constraints combine cardinality (the maximum: 1:1, 1:N, M:N) and participation (the minimum: mandatory or optional) — two genuinely independent questions.